

ตอนที่ 3 — วันที่ AI พุดไม่จบประโยค (21-22 สิงหาคม 2569)

ต่อจาก 2026-07-29(teach mk).md คำศัพท์พื้นฐาน (token, LoRA, VRAM, GGUF, visual token) อธิบายไว้แล้วในตอนที่ 1-2 ตรงนี้จะอ้างถึงเฉยๆ ไม่อธิบายซ้ำ

สรุปสั้นที่สุดก่อนอ่านยาว: โมเดล t02 อ่านหน้าแปลนคานแล้ว **ตอบ JSON ไม่จบ** — พิมพ์วนไปเรื่อยๆ จนหมดโควตาแล้วหยุดกลางประโยค ผลคือทั้งหน้าใช้ไม่ได้เลย แม้แต่ส่วนที่อ่านถูกต้องตอนต้น คืบนี้แก้จบด้วยเทคนิคชื่อ **grammar-constrained decoding** → หน้าที่ผ่านมาเพิ่มจาก **57.9%** เป็น **96.8%** แต่การแก้ครั้งนี้เปิดโปงปัญหาที่ใหญ่กว่า: **ตอบถูกไวยากรณ์ ไม่ได้แปลว่าตอบถูกเนื้อหา**

1. อาการ — "ตอบไม่จบประโยค" หน้าตาเป็นยังไง

เราสั่ง t02 ให้อ่านหน้าแปลนคานของบ้าน 08 สิ่งที่ได้กลับมาคือ:

```
{ "png": "20", "sheet_code": "S-03",  
  "elements": [  
    { "element_id": "B2", "grid_ref_start": "D1", "grid_ref_end": "D2" },  
    { "element_id": "B2", "grid_ref_start": "D2", "grid_ref_end": "D3" },  
    { "element_id": "B2", "grid_ref_start": "D3", "grid_ref_end": "D4" },  
    { "element_id": "B2", "grid_ref_start": "D4", "grid_ref_end": "D5" },  
    ... (วนแบบนี้ไปอีกเป็นร้อยบรรทัด) ...
```

แล้วก็จบแค่นั้น — ไม่มี `]` ไม่มี `}` ปิดท้าย

ทำไมแค่วงเล็บไม่ปิดถึงเป็นเรื่องใหญ่

JSON เป็นภาษาที่เข้มมาก ถ้าวางเล็บไม่ครบ คอมไพเลอร์จะอ่าน**ทั้งไฟล์**ไม่ออก ไม่ใช่อ่านได้บางส่วน

เปรียบเทียบ: เหมือนเขียนจดหมายยาว 3 หน้า แต่ลืมปิดเครื่องหมายคำพูดตั้งแต่บรรทัดแรก คนอ่านยังพออ่านรู้เรื่อง แต่**เครื่อง**จะบอกว่า "ไฟล์นี้เสีย" แล้วทิ้งทั้งฉบับ

ผลจริงที่วัดได้: หน้าแปลนคานหน้านั้นมีคานจริง 31 ตัว โมเดลอ่านถูกไปแล้วหลายตัวตอนต้น แต่เพราะปิดวงเล็บไม่ทัน **เราได้อัน 0 ตัว** — งานที่ทำถูกแล้วก็หายไปด้วย

2. ทำไม AI ถึงวนพูดไม่จบ

AI ไม่ได้ "เขียนคำตอบ" มันเดาคำถัดไปที่ละคำ

นี่คือหัวใจที่ต้องเข้าใจก่อน AI ไม่ได้คิดคำตอบทั้งก้อนแล้วพิมพ์ออกมา มันทำงานแบบนี้: ดูทุกอย่างที่พิมพ์ไปแล้ว → **เดาว่าคำถัดไปน่าจะเป็นอะไร** → พิมพ์ → วนกลับไปข้อ 1

ดังนั้นเวลามันพิมพ์ { "element_id": "B2", "grid_ref_start": "D3"... มันไม่ได้รู้ว่า "ฉันกำลังเขียนคนตัวที่ 40" มันแค่เห็นว่าข้างบนมีแพทเทิร์นแบบนี้ซ้ำๆ แล้วคำที่ "น่าจะตามมา" ก็คือแพทเทิร์นเดิมอีกรอบ

กับดัก: ยิ่งวน ยิ่งมั่นใจว่าต้องวนต่อ

พอมันพิมพ์แพทเทิร์นเดิม 5 รอบ หลักฐานที่มันเห็นก็คือ "หน้านี้มีแพทเทิร์นนี้เยอะมาก" รอบที่ 6 จึงยิ่งมั่นใจกว่าเดิม รอบที่ 20 ยิ่งมั่นใจไปอีก — **มันขุดหลุมให้ตัวเองลึกขึ้นเรื่อยๆ**

เรียกอาการนี้ว่า **repetition loop (การวนซ้ำ)** เป็นโรคประจำตัวของ AI ทุกตัว ไม่ใช่แค่ตัวเรา และไม่ใช่เพราะทูนผิด

แล้วทำไมไม่หยุดเอง

มันจะหยุดต่อเมื่อเดาได้ว่า "คำถัดไปคือสัญญาณจบ" แต่ตอนที่มันติดหลุมแล้ว สัญญาณจบมีคะแนนต่ำกว่าแพทเทิร์นซ้ำเสมอ สุดท้ายเลยชนเพดาน `max_new_tokens` (โควตาจำนวนค่าสูงสุดที่เราตั้งไว้ = 3,000) แล้วโดนตัดกลางคัน

เพดานนี้คือคนตัดจบ ไม่ใช่ AI — มันเลยจบแบบไม่ปดวงเล็บ

3. ความพยายามที่ 1 — ปรับพารามิเตอร์ (ช่วยได้ แต่ไม่พอ)

วันที่ 21 เราใส่ 2 ตัวนี้เข้าไปในคำสั่งสร้างคำตอบ:

พารามิเตอร์	ค่า	ทำอะไร
<code>repetition_penalty</code>	1.15	ค่าที่พิมพ์ไปแล้ว จะถูก หักคะแนน ทำให้โอกาสถูกเลือกซ้ำน้อยลง
<code>no_repeat_ngram_size</code>	8	ห้ามพิมพ์ชุดค่า 8 คำติดที่ เคยพิมพ์เป๊ะๆ มาแล้ว ซ้ำอีก

ได้ผลบ้าง — จาก "ฟังทุกครั้ง" กลายเป็น "ฟังบางครั้ง" แต่ยังไม่ใช่การแก้ที่ต้นเหตุ เพราะทั้งสองตัวนี้แค่**ทำให้การวนยากขึ้น** ไม่ได้ห้ามไม่ให้จบแบบไม่ปดวงเล็บ

4. ความพยายามที่ 2 — Grammar-constrained decoding (ตัวที่แก้จบ)

แนวคิด: ใส่ "ยามเฝ้าประตู" ไว้ทุกครั้งที่พิมพ์

ปกติ AI เลือกคำถัดไปจากคำศัพท์ทั้งหมดที่มี (~151,000 คำ) ได้อย่างอิสระ

grammar-constrained decoding คือการเอากฎไวยากรณ์ของ JSON มาเป็นยาม — ทุกครั้งที่ AI จะพิมพ์คำถัดไป ยามจะดูก่อนว่า "ถ้าพิมพ์คำนี้ JSON จะยังถูกไวยากรณ์อยู่ไหม" ถ้าไม่ ตัดคำนั้นทิ้งทันที ไม่ให้เลือกเลย

เปรียบเทียบง่ายๆ: เหมือนเขียนบนกระดาษที่มีเส้นบังคับ — ปากกาจะเขียนนอกเส้นไม่ได้เลย ไม่ใช่ "เขียนนอกได้แต่โดนหักคะแนน"

ผลที่ตามมาโดยอัตโนมัติ

- วงเล็บปิดครบเสมอ — เพราะถ้าเปิด [ไว้ ยามจะไม่ยอมให้จบจนกว่าจะมี]
- ไม่มี string ที่ลืมปิดัญประกาศ
- ไม่มีตัวเลขพิมพ์ค้าง เช่น 3. แล้วจบ

สำคัญ: นี่คือความต่างระหว่าง "ขอร้อง" กับ "บังคับ" repetition_penalty คือขอร้อง (หักคะแนน แต่ยังทำได้) grammar คือบังคับ (ทำไม่ได้เลย เป็นไปไม่ได้ทางกายภาพ)

เราลองไป 2 ตัว เลือกตัวหลัง

ตัว	ผล
lm-format-enforcer	ใช้ได้ แต่มีจุดอ่อน — ดูข้อ 5
xgrammar	ดีกว่าทุกข้อ → เลือกตัวนี้เป็นค่าเริ่มต้น

5. บทเรียนที่แพงที่สุดของคืนนี้ — ผมวินิจฉัยผิด แล้วเขียนลงเอกสารไปแล้ว

ตรงนี้สำคัญว่าเรื่องเทคนิคทั้งหมดข้างบน มะขามอ่านซ้ำๆ นะ

เกิดอะไรขึ้น

ใส่ grammar เข้าไปแล้ว รันครั้งแรก → ยังฟัง JSON ยัง parse ไม่ผ่าน ผมสรุปทันทีว่า "Unsloth (ตัวโหลดโมเดล) คงกลืนคำสั่ง grammar ทิ้ง ไม่ได้ส่งต่อ" แล้วเขียนข้อสรุปนั้นลง SUMMARY.md ไปเรียบร้อยแล้ว

ผิดครับ ผิดสนิท

ความจริงคืออะไร

พอไล่อ่านโค้ดจริงทีละชั้น (ไม่ใช่เดา) พบว่า:

1. Unsloth ส่งคำสั่งต่อครบถ้วน ไม่ได้กลืนอะไรเลย
2. ตัว grammar ทำงานอยู่ตลอดเวลา
3. **ต้นเหตุจริง:** lm-format-enforcer มองว่า {"a":1,} (มีลูกน้ำเกินก่อนปิด) เป็นสิ่งที่ยอมรับได้ แต่ Python มองว่าผิด — สองฝ่ายนิยามคำว่า "ถูกต้อง" ไม่ตรงกัน

พิสูจน์ได้ใน 1 บรรทัด: เอาไฟล์ที่ "พัง" มาลบลูกน้ำเกินออก → **อ่านได้ทันที** แปลว่า grammar ทำงานถูกมาตลอด ผมกล่าวหาผู้บริสุทธิ์

สิ่งที่ต้องจำ 2 ข้อ

- ข้อ 1 — อย่าโทษเครื่องมือก่อนอ่านโค้ดจริง** "น่าจะเป็นเพราะ X" ที่ฟังดูสมเหตุสมผล $\neq$ "เป็นเพราะ X" เวลาหาสาเหตุ ต้องไล่ดูของจริงทีละชั้น ไม่ใช่เลือกผู้ต้องสงสัยที่เล่าเป็นเรื่องราวได้สวยที่สุด
- ข้อ 2 — เดาผิดแล้วเขียนลงเอกสาร แยกว่าไม่เขียน** ถ้าหลักฐานยังไม่พอชี้ ให้เขียนว่า "ยังไม่รู้สาเหตุ ผู้ต้องสงสัยคือ A หรือ B" อย่าเขียนฟันธงผิดๆ เพราะคนที่มาอ่านทีหลัง (รวมทั้งตัวเราเองอีก 3 เดือน) จะเชื่อมั่นแล้วเดินผิดทางต่อ

เรื่องนี้ถูกบันทึกเป็น บทเรียนข้อ 14 ใน rule_of_tune.md แล้ว

6. ผลลัพธ์จริง — วัดได้เป็นตัวเลข

รันจริงบนการ์ด RTX PRO 6000 (96GB) บ้าน 08 ครบ 25 หน้า + บ้าน 09 อีก 6 หน้า:

	ก่อน (ไม่มี grammar)	หลัง (xgrammar)
หน้าที่ JSON อ่านได้	57.9% (11 จาก 19)	96.8% (30 จาก 31)
หน้าแปลนคานบ้าน 08	✗ พังทั้ง 2 รอบ	✓ ผ่าน

7. บั๊กที่ grammar แก้ไม่ได้ (มี 1 ตัว เจอจริง)

หน้า 07 ของบ้าน 08 ยังพัง โมเดลพิมพ์แบบนี้:

```
"description": "ct_igh_igh_igh_ide_igh_igh_igh_ida_igh_igh_igh_ife_igh_igh..."
```

วนอยู่ข้างในเครื่องหมายคำพูด แล้วชนเพดานคำ

ทำไมยามจับไม่ได้: ข้างใน string ตัวอักษรอะไรก็ถูกไวยากรณ์ทั้งหมด คำว่า igh ก็ถูก บ้าน ก็ถูก อะไรก็ถูก — ยามมีหน้าที่ดูไวยากรณ์ ไม่ได้ดูความหมาย

ทำไม no_repeat_ngram_size=8 ก็จับไม่ได้: มันสลับตัวกันไปเรื่อย (_ide, _ida, _ife) ทำให้ไม่มีชุด 8 คำไหนซ้ำเป๊ะๆ เลย — หลบกฎได้พอดี

วิธีแก้ที่ใช้: ตรวจว่าไฟล์ parse ไม่ผ่าน → สร้างใหม่หน้านั้นหน้าเดียว (1 หน้าจาก 31 ไม่คุ้มที่จะไปนั่งแก้ให้ซับซ้อนกว่านี้)

8. เรื่องที่ใหญ่กว่านี้ — ตอบถูกไวยากรณ์ ≠ ตอบถูกเนื้อหา

พอ JSON อ่านได้แล้ว เราถึงเห็นของจริงเป็นครั้งแรก และของจริงคือยัง**ไม่ดี**

หน้าแปลนคานบ้าน 09 ที่ควรมีคาน 48 ตัว โมเดลตอบมา **2 ตัว** แล้วเขียนมั่วๆ ว่า "F-F x 1'-2', F-F x 2'-3', ...". ยัดทั้งหน้าไว้ในช่องเดียว

เราเอาคำตอบไปวาดทับแบบจริง (ไฟล์ `_overlay_gt_vs_ai_หน้า26.png` ที่ผมส่งให้ดู): - **ซ้าย (เขียว) = เฉลยที่คนตรวจแล้ว** → เส้นทาบแนวคานจริงเป๊ะ 45 จาก 48 ตัว - **ขวา (ส้ม) = AI** → เป็นกล่องลอยๆ ไม่เกาะแนวคานเลย

นี่คือภาพที่อธิบายทุกอย่างในรูปเดียว ตาเปล่าดูก็รู้ว่าคนละเรื่องกัน

ทำไมถึงเป็นแบบนี้ — คำตอบอยู่ที่จำนวนตัวอย่าง

ผมไปนับข้อมูลที่ใช้ทุน t02 จริงๆ ได้แบบนี้:

ประเภทหน้า	t02 เคยเห็นตอนเทรน	ผลลัพธ์
ตารางประตู-หน้าต่าง	~24 หน้า	อ่านได้ 86% ✓
รายการวัสดุ	~158 หน้า	ใช้ได้
แปลนคาน	~13 หน้า	อ่านได้ 11% ✗

เห็นความสัมพันธ์ไหมครับ — หน้าที่มันเห็นเยอะ มันทำได้ดี หน้าที่เห็นน้อย มันทำไม่ได้ ไม่ใช่เพราะแปลนคานยากเกินไป แต่เพราะเราสอนมันแค่ **13 ใบ**

แถมแปลนคานยังเป็นหน้าที่ยากที่สุดด้วย (คานเฉลี่ย 25 ตัวต่อหน้า แต่ละตัวต้องบอกจุดเริ่ม-จุดจบให้ถูก) ยากที่สุด + ตัวอย่างน้อยที่สุด = พังที่สุด

9. แล้วต้องมีข้อมูลกี่หลังถึงจะพอ

งานแบบนี้ต้องการราว **100+** ตัวอย่างต่อประเภทหน้า ถึงจะเริ่มนิ่ง

ระดับ	จำนวนบ้าน	แปลนคานที่ได้
ตอนนี้	11 หลัง (ใช้ทุนจริง 4)	35 ใบ (เห็นจริง 13)
เป้า	25-30 หลัง	80-96 ใบ
เต็มที่	40-50 หลัง	128-160 ใบ

แต่ห้ามทำเพิ่มแบบเดิม — ตรงนี้ประหยัดแรงได้มหาศาล

ผมนับเจอว่า หน้า "รายการวัสดุ" กินแรงไป 435 ไฟล์ = 37% ของงานทั้งหมดที่ทำมา แต่ให้ข้อมูล **elements = 0** (มันเป็นตาราง ไม่มีตำแหน่งชิ้นส่วน)

ถ้าบ้านใหม่ทำเฉพาะหน้าที่ระบบเอาไปใช้จริง จะเหลือ ~38 หน้าต่อหลัง แทนที่จะเป็น ~107 หน้า

เพิ่ม 20 หลัง = ทำ ~750 หน้า (ไม่ใช่ 2,140 หน้า) — ประหยัดแรง 65% ได้ผลเท่ากัน

รายละเอียดเดิมอยู่ที่ `tune_ai/t03/dataset_sizing.md`

10. สรุปบทเรียนครั้งนี้

1. **AI ตอบไม่จบ ≠ AI โง่** — มันติดหลุมวนซ้ำ เพราะมันเดาคำถัดไปที่ละคำ ไม่ได้วางแผนทั้งคำตอบ
2. **"ขอร้อง" กับ "บังคับ" ต่างกัน** — `repetition_penalty` คือขอร้อง `grammar` คือบังคับ ปัญหาโครงสร้างต้องใช้การบังคับ
3. **อย่าฟันธงสาเหตุก่อนอ่านโค้ดจริง** และถ้ายังไม่แน่ใจให้เขียนว่ายังไม่แน่ — เดาคิดที่เขียนลงเอกสารแล้วอันตรายน่าจะไม่เขียน
4. **แก้รูปแบบไม่ได้แก้เนื้อหา** — `grammar` ทำให้ไฟล์อ่านได้ 96.8% แต่ความแม่นยำยังเท่าเดิม
5. **จำนวนตัวอย่างคือตัวกำหนดความแม่นยำ** — 13 ใบได้ 11%, 24 ใบได้ 86% ตัวเลขมันฟ้องตรงๆ
6. **ทำข้อมูลเพิ่มต้องเลือกทำ** — ทำทุกหน้าเท่ากันคือเสียแรง 65% ไปกับหน้าที่ไม่มีใครใช้

คำศัพท์เพิ่มจากตอนนี้

Grammar-constrained decoding (การบังคับไวยากรณ์ตอนสร้างคำตอบ) เทคนิคที่ห้าม AI พิมพ์คำที่จะทำให้ผลลัพธ์ผิดไวยากรณ์ ตัดตัวเลือกนั้นทิ้งก่อนมันจะเลือกได้

xgrammar โปรแกรมที่ทำหน้าที่ยามข้างบน ตัวที่เราเลือกใช้ ดีตรงที่ผลิตลูกน้ำเกินไม่ได้เลยตั้งแต่ต้นทาง

Repetition loop (การวนซ้ำ) อาการที่ AI พิมพ์แพทเทิร์นเดิมซ้ำไม่หยุด เพราะยังพิมพ์ซ้ำ หลักฐานยังบอกให้พิมพ์ซ้ำต่อ

max_new_tokens (เพดานจำนวนคำ) โควต่ำค่าสูงสุดต่อ 1 คำตอบ ของเรที่ตั้งไว้ 3,000 ถ้าชนเพดานจะโดนตัดกลางประโยคทันที

parse (การอ่านไฟล์ให้เป็นข้อมูล) ขั้นตอนที่เครื่องแปลงข้อความเป็นข้อมูลที่ใช้งานต่อได้ ถ้าไวยากรณ์ผิดแม้จุดเดียว จะล้มทั้งไฟล์

recall (ความครบ) วัดว่าของจริงมี 100 ชิ้น AI หาเจอกี่ชิ้น — คนละเรื่องกับ "ที่หาเจอถูกไหม"

Ground truth (เฉลย) ข้อมูลที่คนตรวจแล้วว่าถูก ใช้เป็นไม้บรรทัดวัดว่า AI ตอบถูกแค่ไหน

บันทึกจากรอบเข้าการดจอ คืบวันที่ 21 ต่อเข้าวันที่ 22 สิงหาคม 2569 ไฟล์ที่เกี่ยวข้อง:

`tune_ai/t02/infer_results/SUMMARY.md` · `tune_ai/t03/dataset_sizing.md` ·

`No_touch_box/docs/rule_of_tune.md` ข้อ 13-14